COMPLETE SOURCE CODE EXPLANATION

Controllers & UI Architecture

Line-by-Line Explanation of Presentation & View Controllers

Comprehensive walkthrough of all 14 JavaFX Controller classes. Explains @FXML controls, event-driven action methods, table data binding, and user navigation.

Project: Restaurant Management System

Language: Java 17 / JavaFX / JDBC

Scope: Detailed Line-by-Line Breakdown

Table of Contents

BillingController.java	src\main\java\com\restaurant\controller\BillingController.java
CustomerController.java	src\main\java\com\restaurant\controller\CustomerController.java
DashboardController.java	src\main\java\com\restaurant\controller\DashboardController.java
DashboardHomeController.java	src\main\java\com\restaurant\controller\DashboardHomeController.java
InventoryController.java	src\main\java\com\restaurant\controller\InventoryController.java
KitchenController.java	src\main\java\com\restaurant\controller\KitchenController.java
LoginController.java	src\main\java\com\restaurant\controller\LoginController.java
MenuController.java	src\main\java\com\restaurant\controller\MenuController.java
OrderController.java	src\main\java\com\restaurant\controller\OrderController.java
PaymentController.java	src\main\java\com\restaurant\controller\PaymentController.java
ReportController.java	src\main\java\com\restaurant\controller\ReportController.java
ReservationController.java	src\main\java\com\restaurant\controller\ReservationController.java
SupplierController.java	src\main\java\com\restaurant\controller\SupplierController.java
TableController.java	src\main\java\com\restaurant\controller\TableController.java

BillingController.java

Path: src\main\java\com\restaurant\controller\BillingController.java | Total Lines: 107 | Logical Blocks: 8

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Billing. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–14 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.OrderStatus;
4   import com.restaurant.model.Order;
5   import com.restaurant.service.BillingService;
6   import com.restaurant.service.OrderService;
7   import com.restaurant.util.AlertUtil;
8   import com.restaurant.util.CurrencyUtil;
9   import javafx.collections.FXCollections;
10  import javafx.collections.ObservableList;
11  import javafx.fxml.FXML;
12  import javafx.scene.control.*;
13  import javafx.scene.control.cell.PropertyValueFactory;
14
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 14. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 15–15 Class Declaration & Heritage

```
15  public class BillingController {
```

Architecture Role	Defines BillingController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 15, allowing cross-package invocation.

Lines 16–38 Class Members & Implementation

```
16
17      @FXML private TableView<Order> readyOrdersTable;
18      @FXML private TableColumn<Order, String> colOrderNumber;
19      @FXML private TableColumn<Order, String> colTable;
20      @FXML private TableColumn<Order, String> colCustomer;
21      @FXML private TableColumn<Order, OrderStatus> colStatus;
22      @FXML private TableColumn<Order, Double> colAmount;
23
24      @FXML private TextField discountField;
25      @FXML private TextArea receiptPreviewArea;
26
27      @FXML private Label lblSubtotal;
28      @FXML private Label lblDiscount;
29      @FXML private Label lblTax;
30      @FXML private Label lblService;
31      @FXML private Label lblTotal;
32
33      private final OrderService orderService = new OrderService();
34      private final BillingService billingService = new BillingService();
35      private final ObservableList<Order> billableOrders = FXCollections.observableArrayList();
36      private Order selectedOrder;
37
38      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 16–38 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–56 Method: initialize()

```
39      public void initialize() {
40          colOrderNumber.setCellValueFactory(new PropertyValueFactory<>("orderNumber"));
41          colTable.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
42          colCustomer.setCellValueFactory(new PropertyValueFactory<>("customerName"));
43          colStatus.setCellValueFactory(new PropertyValueFactory<>("status"));
44          colAmount.setCellValueFactory(new PropertyValueFactory<>("totalAmount"));
45
46          readyOrdersTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal)
-> {
47              if (newVal != null) {
48                  // Fetch full order with items
49                  selectedOrder = orderService.getOrderById(newVal.getOrderId());
50                  updateBillPreview();
51              }
52          });
53
54          loadBillableOrders();
55      }
56
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 39–56 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 57–67 Method: loadBillableOrders()

```
57      private void loadBillableOrders() {
58          billableOrders.clear();
59          for (Order o : orderService.getAllOrders()) {
60              if (o.getStatus() == OrderStatus.SERVED || o.getStatus() == OrderStatus.READY || o.getStat
us() == OrderStatus.CONFIRMED) {
61                  billableOrders.add(o);
62              }
63          }
64          readyOrdersTable.setItems(billableOrders);
65      }
66
67      @FXML
```

Component	Method: loadBillableOrders()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 57–67 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–85 Method: handleApplyDiscount()

```
68      public void handleApplyDiscount() {
69          if (selectedOrder == null) {
70              AlertUtil.showWarning("No Selection", "Please select an order to apply discount.");
71              return;
72          }
73          try {
74              double discount = Double.parseDouble(discountField.getText().trim());
75              if (discount < 0 || discount > selectedOrder.getSubtotal()) {
76                  AlertUtil.showError("Invalid Discount", "Discount cannot be negative or greater than s
ubtotal.");
77                  return;
78              }
79              selectedOrder.setDiscountAmount(discount);
80              updateBillPreview();
81          } catch (NumberFormatException e) {
82              AlertUtil.showError("Invalid Number", "Please enter a valid numeric discount amount.");
83          }
84      }
85
```

Component	Method: handleApplyDiscount()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 68–85 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 86–99 Method: updateBillPreview()

```
86      private void updateBillPreview() {
87          if (selectedOrder != null) {
88              lblSubtotal.setText(CurrencyUtil.format(selectedOrder.getSubtotal()));
89              lblDiscount.setText(CurrencyUtil.format(selectedOrder.getDiscountAmount()));
90              lblTax.setText(CurrencyUtil.format(selectedOrder.getTaxAmount()));
91              lblService.setText(CurrencyUtil.format(selectedOrder.getServiceCharge()));
92              lblTotal.setText(CurrencyUtil.format(selectedOrder.getTotalAmount()));
93
94              String receipt = billingService.generateBillReceipt(selectedOrder);
95              receiptPreviewArea.setText(receipt);
96          }
97      }
98
99      @FXML
```

Component	Method: updateBillPreview()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 86–99 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 100–107` Method: handlePrintBill()

```
100       public void handlePrintBill() {
101           if (selectedOrder == null) {
102               AlertUtil.showWarning("No Order", "Select an order to generate bill.");
103               return;
104           }
105           AlertUtil.showInfo("Invoice Generated", "Invoice successfully generated for Order #" + selecte
dOrder.getOrderNumber() + ".\nReady for cashier settlement.");
106       }
107   }
```

Component	Method: handlePrintBill()
Line-by-Line Breakdown	Lines 100–107 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

CustomerController.java

Path: src\main\java\com\restaurant\controller\CustomerController.java | Total Lines: 147 | Logical Blocks: 10

📘 COMPONENT OVERVIEW & RESPONSIBILITY

JavaFX UI Controller for Customer. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.model.Customer;
4   import com.restaurant.service.CustomerService;
5   import com.restaurant.util.AlertUtil;
6   import javafx.collections.FXCollections;
7   import javafx.collections.ObservableList;
8   import javafx.fxml.FXML;
9   import javafx.scene.control.TableColumn;
10  import javafx.scene.control.TableView;
11  import javafx.scene.control.TextField;
12  import javafx.scene.control.cell.PropertyValueFactory;
13
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public class CustomerController {
```

Architecture Role	Defines CustomerController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–34 Class Members & Implementation

```
15
16      @FXML private TableView<Customer> customerTable;
17      @FXML private TableColumn<Customer, Integer> colId;
18      @FXML private TableColumn<Customer, String> colName;
19      @FXML private TableColumn<Customer, String> colPhone;
20      @FXML private TableColumn<Customer, String> colEmail;
21      @FXML private TableColumn<Customer, Integer> colPoints;
22      @FXML private TableColumn<Customer, Double> colSpent;
23
24      @FXML private TextField nameField;
25      @FXML private TextField phoneField;
26      @FXML private TextField emailField;
27      @FXML private TextField addressField;
28      @FXML private TextField searchField;
29
30      private final CustomerService customerService = new CustomerService();
31      private final ObservableList<Customer> customerList = FXCollections.observableArrayList();
32      private Customer selectedCustomer;
33
34      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 15–34 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–55 Method: initialize()

```
35      public void initialize() {
36          colId.setCellValueFactory(new PropertyValueFactory<>("customerId"));
37          colName.setCellValueFactory(new PropertyValueFactory<>("name"));
38          colPhone.setCellValueFactory(new PropertyValueFactory<>("phone"));
39          colEmail.setCellValueFactory(new PropertyValueFactory<>("email"));
40          colPoints.setCellValueFactory(new PropertyValueFactory<>("loyaltyPoints"));
41          colSpent.setCellValueFactory(new PropertyValueFactory<>("totalSpent"));
42
43          customerTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) ->
{
44              if (newVal != null) {
45                  selectedCustomer = newVal;
46                  nameField.setText(newVal.getName());
47                  phoneField.setText(newVal.getPhone());
48                  emailField.setText(newVal.getEmail());
49                  addressField.setText(newVal.getAddress());
50              }
51          });
52
53          loadCustomers();
54      }
55
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 35–55 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–62 Method: loadCustomers()

```
56      private void loadCustomers() {
57          customerList.clear();
58          customerList.addAll(customerService.getAllCustomers());
59          customerTable.setItems(customerList);
60      }
61
62      @FXML
```

Component	Method: loadCustomers()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 56–62 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–81

Method: handleAddCustomer()

```
63      public void handleAddCustomer() {
64          try {
65              boolean success = customerService.registerCustomer(
66                      nameField.getText(),
67                      phoneField.getText(),
68                      emailField.getText(),
69                      addressField.getText()
70              );
71              if (success) {
72                  AlertUtil.showInfo("Success", "Customer registered successfully!");
73                  clearForm();
74                  loadCustomers();
75              }
76          } catch (Exception e) {
77              AlertUtil.showError("Registration Failed", e.getMessage());
78          }
79      }
80
81      @FXML
```

Component	Method: handleAddCustomer()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 63–81 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 82–104` Method: handleUpdateCustomer()

```
82      public void handleUpdateCustomer() {
83          if (selectedCustomer == null) {
84              AlertUtil.showWarning("No Selection", "Please select a customer to update.");
85              return;
86          }
87          try {
88              selectedCustomer.setName(nameField.getText());
89              selectedCustomer.setPhone(phoneField.getText());
90              selectedCustomer.setEmail(emailField.getText());
91              selectedCustomer.setAddress(addressField.getText());
92
93              boolean success = customerService.updateCustomer(selectedCustomer);
94              if (success) {
95                  AlertUtil.showInfo("Success", "Customer updated successfully!");
96                  clearForm();
97                  loadCustomers();
98              }
99          } catch (Exception e) {
100             AlertUtil.showError("Update Failed", e.getMessage());
101         }
102     }
103
104     @FXML
```

Component	Method: handleUpdateCustomer()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 82–104 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 105–121 Method: handleDeleteCustomer()

```
105       public void handleDeleteCustomer() {
106           if (selectedCustomer == null) {
107               AlertUtil.showWarning("No Selection", "Please select a customer to delete.");
108               return;
109           }
110           boolean confirm = AlertUtil.showConfirmation("Confirm Delete", "Are you sure you want to delet
e customer: " + selectedCustomer.getName() + "?");
111           if (confirm) {
112               boolean success = customerService.deleteCustomer(selectedCustomer.getCustomerId());
113               if (success) {
114                   AlertUtil.showInfo("Success", "Customer removed.");
115                   clearForm();
116                   loadCustomers();
117               }
118           }
119       }
120
121       @FXML
```

Component	Method: handleDeleteCustomer()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 105–121 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 122–138 Method: handleSearch()

```
122       public void handleSearch() {
123           String query = searchField.getText();
124           if (query == null || query.trim().isEmpty()) {
125               customerTable.setItems(customerList);
126               return;
127           }
128           String q = query.toLowerCase().trim();
129           ObservableList<Customer> filtered = FXCollections.observableArrayList();
130           for (Customer c : customerList) {
131               if (c.getName().toLowerCase().contains(q) || c.getPhone().contains(q)) {
132                   filtered.add(c);
133               }
134           }
135           customerTable.setItems(filtered);
136       }
137
138       @FXML
```

Component	Method: handleSearch()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 122–138 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 139–147 Method: clearForm()

```
139       public void clearForm() {
140           selectedCustomer = null;
141           nameField.clear();
142           phoneField.clear();
143           emailField.clear();
144           addressField.clear();
145           customerTable.getSelectionModel().clearSelection();
146       }
147   }
```

Component	Method: clearForm()
Line-by-Line Breakdown	Lines 139–147 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DashboardController.java

Path: src\main\java\com\restaurant\controller\DashboardController.java | Total Lines: 148 | Logical Blocks: 21

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Dashboard. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–20 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.Main;
4   import com.restaurant.enums.UserRole;
5   import com.restaurant.model.User;
6   import com.restaurant.service.AuthenticationService;
7   import com.restaurant.service.ReportService;
8   import com.restaurant.util.AlertUtil;
9   import com.restaurant.util.CurrencyUtil;
10  import javafx.fxml.FXML;
11  import javafx.fxml.FXMLLoader;
12  import javafx.scene.Node;
13  import javafx.scene.control.Button;
14  import javafx.scene.control.Label;
15  import javafx.scene.layout.StackPane;
16
17  import java.util.Map;
18  import java.util.logging.Level;
19  import java.util.logging.Logger;
20
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 20. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 21–21 Class Declaration & Heritage

```
21  public class DashboardController {
```

Architecture Role	Defines DashboardController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 21, allowing cross-package invocation.

Lines 22–24 Class Members & Implementation

```
22      private static final Logger LOGGER = Logger.getLogger(DashboardController.class.getName());
23      private static DashboardController instance;
24
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 22–24 (3 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 25–56 Method: getInstance()

```
25      public static DashboardController getInstance() {
26          return instance;
27      }
28
29      @FXML private Label userGreetingLabel;
30      @FXML private Label userRoleBadge;
31      @FXML private StackPane contentArea;
32
33      // Stat metric labels
34      @FXML private Label statRevenueLabel;
35      @FXML private Label statOrdersLabel;
36      @FXML private Label statTablesLabel;
37      @FXML private Label statKitchenLabel;
38      @FXML private Label statLowStockLabel;
39
40      // Nav buttons
41      @FXML private Button btnDashboard;
42      @FXML private Button btnOrders;
43      @FXML private Button btnKitchen;
44      @FXML private Button btnTables;
45      @FXML private Button btnMenu;
46      @FXML private Button btnCustomers;
47      @FXML private Button btnReservations;
48      @FXML private Button btnInventory;
49      @FXML private Button btnSuppliers;
50      @FXML private Button btnBilling;
51      @FXML private Button btnPayments;
52      @FXML private Button btnReports;
53
54      private final ReportService reportService = new ReportService();
55
56      @FXML
```

Component	Method: getInstance()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 25–56 (32 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 57–67 Method: initialize()

```
57      public void initialize() {
58          instance = this;
59          User user = AuthenticationService.getCurrentUser();
60          if (user != null) {
61              userGreetingLabel.setText("Welcome, " + user.getFullName());
62              userRoleBadge.setText(user.getRole().name());
63              applyRolePermissions(user.getRole());
64          }
65          loadDashboardMetrics();
66      }
67
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 57–67 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–86 Method: applyRolePermissions()

```
68      private void applyRolePermissions(UserRole role) {
69          // Enforce role-based UI access control
70          if (role == UserRole.KITCHEN_STAFF) {
71              if (btnOrders != null) btnOrders.setDisable(true);
72              if (btnBilling != null) btnBilling.setDisable(true);
73              if (btnPayments != null) btnPayments.setDisable(true);
74              if (btnReports != null) btnReports.setDisable(true);
75          } else if (role == UserRole.WAITER) {
76              if (btnBilling != null) btnBilling.setDisable(true);
77              if (btnPayments != null) btnPayments.setDisable(true);
78              if (btnReports != null) btnReports.setDisable(true);
79              if (btnInventory != null) btnInventory.setDisable(true);
80          } else if (role == UserRole.CASHIER) {
81              if (btnMenu != null) btnMenu.setDisable(true);
82              if (btnKitchen != null) btnKitchen.setDisable(true);
83              if (btnInventory != null) btnInventory.setDisable(true);
84          }
85      }
86
```

Component	Method: applyRolePermissions()
Line-by-Line Breakdown	Lines 68–86 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–115 Method: loadDashboardMetrics()

```
87      public void loadDashboardMetrics() {
88          try {
89              Map<String, Object> metrics = reportService.getTodaySummaryMetrics();
90              if (statRevenueLabel != null) {
91                  double rev = (double) metrics.getOrDefault("todayRevenue", 0.0);
92                  statRevenueLabel.setText(CurrencyUtil.format(rev));
93              }
94              if (statOrdersLabel != null) {
95                  int orders = (int) metrics.getOrDefault("todayOrders", 0);
96                  statOrdersLabel.setText(String.valueOf(orders));
97              }
98              if (statTablesLabel != null) {
99                  int avail = (int) metrics.getOrDefault("availableTables", 0);
100                 int occ = (int) metrics.getOrDefault("occupiedTables", 0);
101                 statTablesLabel.setText(avail + " Free / " + occ + " Busy");
102             }
103             if (statKitchenLabel != null) {
104                 int kitchen = (int) metrics.getOrDefault("pendingKitchen", 0);
105                 statKitchenLabel.setText(String.valueOf(kitchen));
106             }
107             if (statLowStockLabel != null) {
108                 int lowStock = (int) metrics.getOrDefault("lowStockCount", 0);
109                 statLowStockLabel.setText(String.valueOf(lowStock));
110             }
111         } catch (Exception e) {
112             LOGGER.log(Level.WARNING, "Error updating dashboard metrics", e);
113         }
114     }
115
```

Component	Method: loadDashboardMetrics()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 87–115 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 116–126 Method: loadView()

```
116       private void loadView(String fxmlFile) {
117           try {
118               FXMLLoader loader = new FXMLLoader(getClass().getResource("/fxml/" + fxmlFile));
119               Node node = loader.load();
120               contentArea.getChildren().setAll(node);
121           } catch (Exception e) {
122               LOGGER.log(Level.SEVERE, "Failed loading view: " + fxmlFile, e);
123               AlertUtil.showError("Navigation Error", "Could not load view: " + fxmlFile + "\n" + e.getM
essage());
124           }
125       }
126
```

Component	Method: loadView()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 116–126 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 127–127 Method: showDashboard()

```
127       @FXML public void showDashboard() { loadDashboardMetrics(); loadView("dashboard_home.fxml"); }
```

Component	Method: showDashboard()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 127–127 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 128–128 Method: showOrders()

```
128       @FXML public void showOrders() { loadView("orders.fxml"); }
```

Component	Method: showOrders()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 128–128 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 129–129` **Method: showKitchen()**

```
129       @FXML public void showKitchen() { loadView("kitchen.fxml"); }
```

Component	Method: showKitchen()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 129–129 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 130–130` **Method: showTables()**

```
130       @FXML public void showTables() { loadView("tables.fxml"); }
```

Component	Method: showTables()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 130–130 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 131–131` **Method: showMenu()**

```
131       @FXML public void showMenu() { loadView("menu.fxml"); }
```

Component	Method: showMenu()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 131–131 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 132–132` **Method: showCustomers()**

```
132       @FXML public void showCustomers() { loadView("customers.fxml"); }
```

Component	Method: showCustomers()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 132–132 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 133–133` Method: showReservations()

```
133       @FXML public void showReservations() { loadView("reservations.fxml"); }
```

Component	Method: showReservations()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 133–133 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 134–134` Method: showInventory()

```
134       @FXML public void showInventory() { loadView("inventory.fxml"); }
```

Component	Method: showInventory()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 134–134 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 135–135` Method: showSuppliers()

```
135       @FXML public void showSuppliers() { loadView("suppliers.fxml"); }
```

Component	Method: showSuppliers()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 135–135 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 136–136` Method: showBilling()

```
136       @FXML public void showBilling() { loadView("billing.fxml"); }
```

Component	Method: showBilling()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 136–136 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 137–137` Method: showPayments()

```
137       @FXML public void showPayments() { loadView("payments.fxml"); }
```

Component	Method: showPayments()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 137–137 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 138–140` Method: showReports()

```
138       @FXML public void showReports() { loadView("reports.fxml"); }
139
140       @FXML
```

Component	Method: showReports()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 138–140 (3 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 141–148` Method: handleLogout()

```
141       public void handleLogout() {
142           boolean confirm = AlertUtil.showConfirmation("Confirm Logout", "Are you sure you want to log o
ut?");
143           if (confirm) {
144               new AuthenticationService().logout();
145               Main.loadLoginScreen();
146           }
147       }
148   }
```

Component	Method: handleLogout()
Line-by-Line Breakdown	Lines 141–148 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DashboardHomeController.java

Path: src\main\java\com\restaurant\controller\DashboardHomeController.java | Total Lines: 71 | Logical Blocks: 7

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for DashboardHome. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.service.ReportService;
4   import com.restaurant.util.CurrencyUtil;
5   import javafx.fxml.FXML;
6   import javafx.scene.control.Label;
7
8   import java.util.Map;
9   import java.util.logging.Level;
10  import java.util.logging.Logger;
11
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class DashboardHomeController {
```

Architecture Role	Defines DashboardHomeController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–23 Class Members & Implementation

```
13      private static final Logger LOGGER = Logger.getLogger(DashboardHomeController.class.getName());
14
15      @FXML private Label statRevenueLabel;
16      @FXML private Label statOrdersLabel;
17      @FXML private Label statTablesLabel;
18      @FXML private Label statKitchenLabel;
19      @FXML private Label statLowStockLabel;
20
21      private final ReportService reportService = new ReportService();
22
23      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 13–23 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 24–28 Method: initialize()

```
24      public void initialize() {
25          loadMetrics();
26      }
27
28      @FXML
```

Component	Method: initialize()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 24–28 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–58

Method: loadMetrics()

```
29      public void loadMetrics() {
30          try {
31              Map<String, Object> metrics = reportService.getTodaySummaryMetrics();
32              if (statRevenueLabel != null) {
33                  double rev = (double) metrics.getOrDefault("todayRevenue", 0.0);
34                  statRevenueLabel.setText(CurrencyUtil.format(rev));
35              }
36              if (statOrdersLabel != null) {
37                  int orders = (int) metrics.getOrDefault("todayOrders", 0);
38                  statOrdersLabel.setText(String.valueOf(orders));
39              }
40              if (statTablesLabel != null) {
41                  int avail = (int) metrics.getOrDefault("availableTables", 0);
42                  int occ = (int) metrics.getOrDefault("occupiedTables", 0);
43                  statTablesLabel.setText(avail + " Free / " + occ + " Busy");
44              }
45              if (statKitchenLabel != null) {
46                  int kitchen = (int) metrics.getOrDefault("pendingKitchen", 0);
47                  statKitchenLabel.setText(String.valueOf(kitchen));
48              }
49              if (statLowStockLabel != null) {
50                  int lowStock = (int) metrics.getOrDefault("lowStockCount", 0);
51                  statLowStockLabel.setText(String.valueOf(lowStock));
52              }
53          } catch (Exception e) {
54              LOGGER.log(Level.WARNING, "Error updating dashboard metrics", e);
55          }
56      }
57
58      @FXML
```

Component	Method: loadMetrics()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 29–58 (30 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–65 Method: handleNewOrder()

```
59      public void handleNewOrder() {
60          if (DashboardController.getInstance() != null) {
61              DashboardController.getInstance().showOrders();
62          }
63      }
64
65      @FXML
```

Component	Method: handleNewOrder()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 59–65 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–71 Method: handleKitchen()

```
66      public void handleKitchen() {
67          if (DashboardController.getInstance() != null) {
68              DashboardController.getInstance().showKitchen();
69          }
70      }
71  }
```

Component	Method: handleKitchen()
Line-by-Line Breakdown	Lines 66–71 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InventoryController.java

Path: src\main\java\com\restaurant\controller\InventoryController.java | Total Lines: 201 | Logical Blocks: 13

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Inventory. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–15 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.model.InventoryItem;
4   import com.restaurant.model.Supplier;
5   import com.restaurant.service.InventoryService;
6   import com.restaurant.service.SupplierService;
7   import com.restaurant.util.AlertUtil;
8   import javafx.collections.FXCollections;
9   import javafx.collections.ObservableList;
10  import javafx.fxml.FXML;
11  import javafx.scene.control.*;
12  import javafx.scene.control.cell.PropertyValueFactory;
13
14  import java.util.List;
15
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 15. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 16–16 Class Declaration & Heritage

```
16  public class InventoryController {
```

Architecture Role	Defines InventoryController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 16, allowing cross-package invocation.

Lines 17–43 Class Members & Implementation

```
17
18      @FXML private TableView<InventoryItem> inventoryTable;
19      @FXML private TableColumn<InventoryItem, Integer> colId;
20      @FXML private TableColumn<InventoryItem, String> colName;
21      @FXML private TableColumn<InventoryItem, String> colCategory;
22      @FXML private TableColumn<InventoryItem, Double> colQty;
23      @FXML private TableColumn<InventoryItem, String> colUnit;
24      @FXML private TableColumn<InventoryItem, Double> colMinStock;
25      @FXML private TableColumn<InventoryItem, Double> colUnitCost;
26      @FXML private TableColumn<InventoryItem, String> colSupplier;
27
28      @FXML private TextField nameField;
29      @FXML private TextField categoryField;
30      @FXML private TextField quantityField;
31      @FXML private TextField unitField;
32      @FXML private TextField minStockField;
33      @FXML private TextField unitCostField;
34      @FXML private ComboBox<Supplier> supplierCombo;
35
36      @FXML private TextField adjustQtyField;
37
38      private final InventoryService inventoryService = new InventoryService();
39      private final SupplierService supplierService = new SupplierService();
40      private final ObservableList<InventoryItem> inventoryList = FXCollections.observableArrayList();
41      private InventoryItem selectedItem;
42
43      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 17–43 (27 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 44–78 Method: initialize()

```
44      public void initialize() {
45          colId.setCellValueFactory(new PropertyValueFactory<>("inventoryId"));
46          colName.setCellValueFactory(new PropertyValueFactory<>("ingredientName"));
47          colCategory.setCellValueFactory(new PropertyValueFactory<>("category"));
48          colQty.setCellValueFactory(new PropertyValueFactory<>("quantity"));
49          colUnit.setCellValueFactory(new PropertyValueFactory<>("unit"));
50          colMinStock.setCellValueFactory(new PropertyValueFactory<>("minimumStock"));
51          colUnitCost.setCellValueFactory(new PropertyValueFactory<>("unitCost"));
52          colSupplier.setCellValueFactory(new PropertyValueFactory<>("supplierName"));
53
54          loadSuppliers();
55
56          inventoryTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) ->
{
57              if (newVal != null) {
58                  selectedItem = newVal;
59                  nameField.setText(newVal.getIngredientName());
60                  categoryField.setText(newVal.getCategory());
61                  quantityField.setText(String.valueOf(newVal.getQuantity()));
62                  unitField.setText(newVal.getUnit());
63                  minStockField.setText(String.valueOf(newVal.getMinimumStock()));
64                  unitCostField.setText(String.valueOf(newVal.getUnitCost()));
65                  if (newVal.getSupplierId() != null) {
66                      for (Supplier s : supplierCombo.getItems()) {
67                          if (s.getSupplierId() == newVal.getSupplierId()) {
68                              supplierCombo.setValue(s);
69                              break;
70                          }
71                      }
72                  }
73              }
74          });
75
76          loadInventory();
77      }
78
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 44–78 (35 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–82 Method: loadSuppliers()

```
79      private void loadSuppliers() {
80          supplierCombo.setItems(FXCollections.observableArrayList(supplierService.getAllSuppliers()));
81      }
82
```

Component	Method: loadSuppliers()
Line-by-Line Breakdown	Lines 79–82 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 83–89 Method: loadInventory()

```
83      private void loadInventory() {
84          inventoryList.clear();
85          inventoryList.addAll(inventoryService.getAllInventory());
86          inventoryTable.setItems(inventoryList);
87      }
88
89      @FXML
```

Component	Method: loadInventory()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 83–89 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 90–118 Method: handleAddItem()

```
90      public void handleAddItem() {
91          try {
92              Supplier s = supplierCombo.getValue();
93              Integer sId = s != null ? s.getSupplierId() : null;
94              double qty = Double.parseDouble(quantityField.getText().trim());
95              double min = Double.parseDouble(minStockField.getText().trim());
96              double cost = Double.parseDouble(unitCostField.getText().trim());
97
98              boolean success = inventoryService.addInventoryItem(
99                      nameField.getText(),
100                     categoryField.getText(),
101                     qty,
102                     unitField.getText(),
103                     min,
104                     cost,
105                     sId
106             );
107
108             if (success) {
109                 AlertUtil.showInfo("Success", "Inventory item added.");
110                 clearForm();
111                 loadInventory();
112             }
113         } catch (Exception e) {
114             AlertUtil.showError("Save Error", e.getMessage());
115         }
116     }
117
118     @FXML
```

Component	Method: handleAddItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 90–118 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 119–145` **Method: handleUpdateItem()**

```
119       public void handleUpdateItem() {
120           if (selectedItem == null) {
121               AlertUtil.showWarning("No Selection", "Select an item to update.");
122               return;
123           }
124           try {
125               Supplier s = supplierCombo.getValue();
126               selectedItem.setIngredientName(nameField.getText());
127               selectedItem.setCategory(categoryField.getText());
128               selectedItem.setQuantity(Double.parseDouble(quantityField.getText().trim()));
129               selectedItem.setUnit(unitField.getText());
130               selectedItem.setMinimumStock(Double.parseDouble(minStockField.getText().trim()));
131               selectedItem.setUnitCost(Double.parseDouble(unitCostField.getText().trim()));
132               selectedItem.setSupplierId(s != null ? s.getSupplierId() : null);
133
134               boolean success = inventoryService.updateInventoryItem(selectedItem);
135               if (success) {
136                   AlertUtil.showInfo("Success", "Inventory updated.");
137                   clearForm();
138                   loadInventory();
139               }
140           } catch (Exception e) {
141               AlertUtil.showError("Update Error", e.getMessage());
142           }
143       }
144
145       @FXML
```

Component	Method: handleUpdateItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 119–145 (27 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 146–162 Method: handleQuickStockAdjust()

```
146       public void handleQuickStockAdjust() {
147           if (selectedItem == null) {
148               AlertUtil.showWarning("No Selection", "Select an item to adjust stock.");
149               return;
150           }
151           try {
152               double addQty = Double.parseDouble(adjustQtyField.getText().trim());
153               double newQty = Math.max(0, selectedItem.getQuantity() + addQty);
154               inventoryService.updateStockQuantity(selectedItem.getInventoryId(), newQty);
155               adjustQtyField.clear();
156               loadInventory();
157           } catch (Exception e) {
158               AlertUtil.showError("Adjustment Error", e.getMessage());
159           }
160       }
161
162       @FXML
```

Component	Method: handleQuickStockAdjust()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 146–162 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 163–176 Method: handleDeleteItem()

```
163       public void handleDeleteItem() {
164           if (selectedItem == null) {
165               AlertUtil.showWarning("No Selection", "Select an item to delete.");
166               return;
167           }
168           boolean confirm = AlertUtil.showConfirmation("Confirm Delete", "Delete " + selectedItem.getIng
redientName() + "?");
169           if (confirm) {
170               inventoryService.deleteInventoryItem(selectedItem.getInventoryId());
171               clearForm();
172               loadInventory();
173           }
174       }
175
176       @FXML
```

Component	Method: handleDeleteItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 163–176 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 177–182 Method: handleShowLowStockOnly()

```
177       public void handleShowLowStockOnly() {
178           List<InventoryItem> low = inventoryService.getLowStockItems();
179           inventoryTable.setItems(FXCollections.observableArrayList(low));
180       }
181
182       @FXML
```

Component	Method: handleShowLowStockOnly()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 177–182 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 183–188 Method: handleShowAllSorted()

```
183       public void handleShowAllSorted() {
184           List<InventoryItem> sorted = inventoryService.getInventorySortedByStockAscending();
185           inventoryTable.setItems(FXCollections.observableArrayList(sorted));
186       }
187
188       @FXML
```

Component	Method: handleShowAllSorted()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 183–188 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 189–201 Method: clearForm()

```
189       public void clearForm() {
190           selectedItem = null;
191           nameField.clear();
192           categoryField.clear();
193           quantityField.clear();
194           unitField.clear();
195           minStockField.clear();
196           unitCostField.clear();
197           adjustQtyField.clear();
198           supplierCombo.getSelectionModel().clearSelection();
199           inventoryTable.getSelectionModel().clearSelection();
200       }
201   }
```

Component	Method: clearForm()
Line-by-Line Breakdown	Lines 189–201 (13 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

KitchenController.java

Path: src\main\java\com\restaurant\controller\KitchenController.java | Total Lines: 105 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Kitchen. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.OrderStatus;
4   import com.restaurant.model.Order;
5   import com.restaurant.model.OrderItem;
6   import com.restaurant.service.KitchenService;
7   import com.restaurant.util.AlertUtil;
8   import javafx.collections.FXCollections;
9   import javafx.collections.ObservableList;
10  import javafx.fxml.FXML;
11  import javafx.scene.control.*;
12  import javafx.scene.control.cell.PropertyValueFactory;
13
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public class KitchenController {
```

Architecture Role	Defines KitchenController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–34 Class Members & Implementation

```
15
16      @FXML private TableView<Order> kitchenOrdersTable;
17      @FXML private TableColumn<Order, Integer> colOrderId;
18      @FXML private TableColumn<Order, String> colTable;
19      @FXML private TableColumn<Order, String> colOrderNumber;
20      @FXML private TableColumn<Order, OrderStatus> colStatus;
21      @FXML private TableColumn<Order, String> colTime;
22
23      @FXML private ListView<String> orderItemsListView;
24      @FXML private TextArea notesArea;
25
26      @FXML private Button btnStartPreparing;
27      @FXML private Button btnMarkReady;
28      @FXML private Button btnMarkServed;
29
30      private final KitchenService kitchenService = new KitchenService();
31      private final ObservableList<Order> kitchenList = FXCollections.observableArrayList();
32      private Order selectedOrder;
33
34      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 15–34 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–51 Method: initialize()

```
35      public void initialize() {
36          colOrderId.setCellValueFactory(new PropertyValueFactory<>("orderId"));
37          colTable.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
38          colOrderNumber.setCellValueFactory(new PropertyValueFactory<>("orderNumber"));
39          colStatus.setCellValueFactory(new PropertyValueFactory<>("status"));
40          colTime.setCellValueFactory(new PropertyValueFactory<>("orderDateTime"));
41
42          kitchenOrdersTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVa
l) -> {
43              if (newVal != null) {
44                  selectedOrder = newVal;
45                  displayOrderDetails(newVal);
46              }
47          });
48
49          loadKitchenOrders();
50      }
51
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 35–51 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 52–66 Method: displayOrderDetails()

```
52      private void displayOrderDetails(Order order) {
53          orderItemsListView.getItems().clear();
54          if (order.getItems() != null) {
55              for (OrderItem item : order.getItems()) {
56                  String desc = item.getQuantity() + "x  " + item.getItemName();
57                  if (item.getSpecialInstructions() != null && !item.getSpecialInstructions().isEmpty())
{
58                      desc += " [Note: " + item.getSpecialInstructions() + "]";
59                  }
60                  orderItemsListView.getItems().add(desc);
61              }
62          }
63          notesArea.setText(order.getSpecialNotes() != null ? order.getSpecialNotes() : "No special inst
ructions.");
64      }
65
66      @FXML
```

Component	Method: displayOrderDetails()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 52–66 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–76 Method: loadKitchenOrders()

```
67      public void loadKitchenOrders() {
68          kitchenList.clear();
69          kitchenList.addAll(kitchenService.getActiveKitchenOrders());
70          kitchenOrdersTable.setItems(kitchenList);
71          orderItemsListView.getItems().clear();
72          notesArea.clear();
73          selectedOrder = null;
74      }
75
76      @FXML
```

Component	Method: loadKitchenOrders()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 67–76 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 77–86 Method: handleStartPreparing()

```
77      public void handleStartPreparing() {
78          if (selectedOrder == null) {
79              AlertUtil.showWarning("No Selection", "Please select an order from the kitchen queue.");
80              return;
81          }
82          kitchenService.markAsPreparing(selectedOrder.getOrderId());
83          loadKitchenOrders();
84      }
85
86      @FXML
```

Component	Method: handleStartPreparing()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 77–86 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–96 Method: handleMarkReady()

```
87      public void handleMarkReady() {
88          if (selectedOrder == null) {
89              AlertUtil.showWarning("No Selection", "Please select an order.");
90              return;
91          }
92          kitchenService.markAsReady(selectedOrder.getOrderId());
93          loadKitchenOrders();
94      }
95
96      @FXML
```

Component	Method: handleMarkReady()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 87–96 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 97–105 Method: handleMarkServed()

```
97      public void handleMarkServed() {
98          if (selectedOrder == null) {
99              AlertUtil.showWarning("No Selection", "Please select an order.");
100             return;
101         }
102         kitchenService.markAsServed(selectedOrder.getOrderId());
103         loadKitchenOrders();
104     }
105 }
```

Component	Method: handleMarkServed()
Line-by-Line Breakdown	Lines 97–105 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

LoginController.java

Path: src\main\java\com\restaurant\controller\LoginController.java | Total Lines: 72 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Login. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–12 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.Main;
4   import com.restaurant.exception.InvalidLoginException;
5   import com.restaurant.model.User;
6   import com.restaurant.service.AuthenticationService;
7   import com.restaurant.util.AlertUtil;
8   import javafx.fxml.FXML;
9   import javafx.scene.control.Label;
10  import javafx.scene.control.PasswordField;
11  import javafx.scene.control.TextField;
12
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 12. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 13–13 Class Declaration & Heritage

```
13  public class LoginController {
```

Architecture Role	Defines LoginController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 13, allowing cross-package invocation.

Lines 14–21 Class Members & Implementation

```
14
15      @FXML private TextField usernameField;
16      @FXML private PasswordField passwordField;
17      @FXML private Label errorLabel;
18
19      private final AuthenticationService authService = new AuthenticationService();
20
21      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 14–21 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 22–28 Method: initialize()

```
22      public void initialize() {
23          if (errorLabel != null) {
24              errorLabel.setText("");
25          }
26      }
27
28      @FXML
```

Component	Method: initialize()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 22–28 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–49 Method: handleLogin()

```
29      public void handleLogin() {
30          String username = usernameField.getText();
31          String password = passwordField.getText();
32
33          try {
34              User user = authService.login(username, password);
35              if (user != null) {
36                  Main.loadMainDashboard();
37              }
38          } catch (InvalidLoginException e) {
39              if (errorLabel != null) {
40                  errorLabel.setText(e.getMessage());
41              } else {
42                  AlertUtil.showError("Login Failed", e.getMessage());
43              }
44          } catch (Exception e) {
45              AlertUtil.showError("System Error", "Database or connection error: " + e.getMessage());
46          }
47      }
48
49      @FXML
```

Component	Method: handleLogin()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 29–49 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 50–55 Method: fillAdminDemo()

```
50      public void fillAdminDemo() {
51          usernameField.setText("admin");
52          passwordField.setText("admin123");
53      }
54
55      @FXML
```

Component	Method: fillAdminDemo()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 50–55 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–61 Method: fillWaiterDemo()

```
56      public void fillWaiterDemo() {
57          usernameField.setText("waiter1");
58          passwordField.setText("waiter123");
59      }
60
61      @FXML
```

Component	Method: fillWaiterDemo()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 56–61 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–67 Method: fillCashierDemo()

```
62      public void fillCashierDemo() {
63          usernameField.setText("cashier1");
64          passwordField.setText("cashier123");
65      }
66
67      @FXML
```

Component	Method: fillCashierDemo()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 62–67 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–72 Method: fillChefDemo()

```
68      public void fillChefDemo() {
69          usernameField.setText("chef1");
70          passwordField.setText("kitchen123");
71      }
72  }
```

Component	Method: fillChefDemo()
Line-by-Line Breakdown	Lines 68–72 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

MenuController.java

Path: src\main\java\com\restaurant\controller\MenuController.java | Total Lines: 194 | Logical Blocks: 12

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Menu. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–14 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.model.MenuCategory;
4   import com.restaurant.model.MenuItem;
5   import com.restaurant.service.MenuService;
6   import com.restaurant.util.AlertUtil;
7   import javafx.collections.FXCollections;
8   import javafx.collections.ObservableList;
9   import javafx.fxml.FXML;
10  import javafx.scene.control.*;
11  import javafx.scene.control.cell.PropertyValueFactory;
12
13  import java.util.List;
14
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 14. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 15–15 Class Declaration & Heritage

```
15  public class MenuController {
```

Architecture Role	Defines MenuController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 15, allowing cross-package invocation.

Lines 16–39 Class Members & Implementation

```
16
17      @FXML private TableView<MenuItem> menuTable;
18      @FXML private TableColumn<MenuItem, Integer> colId;
19      @FXML private TableColumn<MenuItem, String> colCategory;
20      @FXML private TableColumn<MenuItem, String> colName;
21      @FXML private TableColumn<MenuItem, Double> colPrice;
22      @FXML private TableColumn<MenuItem, Integer> colPrepTime;
23      @FXML private TableColumn<MenuItem, Boolean> colAvailable;
24
25      @FXML private ComboBox<MenuCategory> categoryComboBox;
26      @FXML private TextField nameField;
27      @FXML private TextField priceField;
28      @FXML private TextField prepTimeField;
29      @FXML private TextArea descriptionArea;
30      @FXML private CheckBox availableCheckBox;
31
32      @FXML private TextField searchField;
33      @FXML private ComboBox<MenuCategory> filterCategoryBox;
34
35      private final MenuService menuService = new MenuService();
36      private final ObservableList<MenuItem> menuItemsList = FXCollections.observableArrayList();
37      private MenuItem selectedItem;
38
39      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 16–39 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 40–70` **Method: initialize()**

```
40      public void initialize() {
41          colId.setCellValueFactory(new PropertyValueFactory<>("itemId"));
42          colCategory.setCellValueFactory(new PropertyValueFactory<>("categoryName"));
43          colName.setCellValueFactory(new PropertyValueFactory<>("name"));
44          colPrice.setCellValueFactory(new PropertyValueFactory<>("price"));
45          colPrepTime.setCellValueFactory(new PropertyValueFactory<>("preparationTime"));
46          colAvailable.setCellValueFactory(new PropertyValueFactory<>("available"));
47
48          loadCategories();
49
50          menuTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {
51              if (newVal != null) {
52                  selectedItem = newVal;
53                  nameField.setText(newVal.getName());
54                  priceField.setText(String.valueOf(newVal.getPrice()));
55                  prepTimeField.setText(String.valueOf(newVal.getPreparationTime()));
56                  descriptionArea.setText(newVal.getDescription());
57                  availableCheckBox.setSelected(newVal.isAvailable());
58
59                  for (MenuCategory cat : categoryComboBox.getItems()) {
60                      if (cat.getCategoryId() == newVal.getCategoryId()) {
61                          categoryComboBox.setValue(cat);
62                          break;
63                      }
64                  }
65              }
66          });
67
68          loadMenuItems();
69      }
70
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 40–70 (31 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 71–81 Method: loadCategories()

```
71      private void loadCategories() {
72          List<MenuCategory> categories = menuService.getAllCategories();
73          categoryComboBox.setItems(FXCollections.observableArrayList(categories));
74
75          ObservableList<MenuCategory> filterList = FXCollections.observableArrayList();
76          filterList.add(new MenuCategory(0, "All Categories", "", 0));
77          filterList.addAll(categories);
78          filterCategoryBox.setItems(filterList);
79          filterCategoryBox.setValue(filterList.get(0));
80      }
81
```

Component	Method: loadCategories()
Line-by-Line Breakdown	Lines 71–81 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 82–88 Method: loadMenuItems()

```
82      private void loadMenuItems() {
83          menuItemsList.clear();
84          menuItemsList.addAll(menuService.getAllMenuItems());
85          menuTable.setItems(menuItemsList);
86      }
87
88      @FXML
```

Component	Method: loadMenuItems()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 82–88 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 89–120` **Method: handleAddItem()**

```
89      public void handleAddItem() {
90          try {
91              MenuCategory cat = categoryComboBox.getValue();
92              if (cat == null) {
93                  AlertUtil.showWarning("Missing Category", "Please select a menu category.");
94                  return;
95              }
96              double price = Double.parseDouble(priceField.getText().trim());
97              int prepTime = Integer.parseInt(prepTimeField.getText().trim());
98
99              boolean success = menuService.addMenuItem(
100                     cat.getCategoryId(),
101                     nameField.getText(),
102                     descriptionArea.getText(),
103                     price,
104                     prepTime,
105                     availableCheckBox.isSelected()
106             );
107
108             if (success) {
109                 AlertUtil.showInfo("Success", "Menu item added successfully.");
110                 clearForm();
111                 loadMenuItems();
112             }
113         } catch (NumberFormatException e) {
114             AlertUtil.showError("Invalid Number", "Price and Prep Time must be valid numeric value
s.");
115         } catch (Exception e) {
116             AlertUtil.showError("Save Failed", e.getMessage());
117         }
118     }
119
120     @FXML
```

Component	Method: handleAddItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 89–120 (32 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 121–146` Method: handleUpdateItem()

```
121        public void handleUpdateItem() {
122            if (selectedItem == null) {
123                AlertUtil.showWarning("No Selection", "Please select an item to edit.");
124                return;
125            }
126            try {
127                MenuCategory cat = categoryComboBox.getValue();
128                if (cat != null) selectedItem.setCategoryId(cat.getCategoryId());
129                selectedItem.setName(nameField.getText());
130                selectedItem.setPrice(Double.parseDouble(priceField.getText().trim()));
131                selectedItem.setPreparationTime(Integer.parseInt(prepTimeField.getText().trim()));
132                selectedItem.setDescription(descriptionArea.getText());
133                selectedItem.setAvailable(availableCheckBox.isSelected());
134
135                boolean success = menuService.updateMenuItem(selectedItem);
136                if (success) {
137                    AlertUtil.showInfo("Success", "Menu item updated.");
138                    clearForm();
139                    loadMenuItems();
140                }
141            } catch (Exception e) {
142                AlertUtil.showError("Update Failed", e.getMessage());
143            }
144        }
145
146        @FXML
```

Component	Method: handleUpdateItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 121–146 (26 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 147–163 Method: handleDeleteItem()

```
147       public void handleDeleteItem() {
148           if (selectedItem == null) {
149               AlertUtil.showWarning("No Selection", "Please select an item to delete.");
150               return;
151           }
152           boolean confirm = AlertUtil.showConfirmation("Confirm Delete", "Delete " + selectedItem.getNam
e() + "?");
153           if (confirm) {
154               boolean success = menuService.deleteMenuItem(selectedItem.getItemId());
155               if (success) {
156                   AlertUtil.showInfo("Success", "Menu item deleted.");
157                   clearForm();
158                   loadMenuItems();
159               }
160           }
161       }
162
163       @FXML
```

Component	Method: handleDeleteItem()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 147–163 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 164–173 Method: handleSearch()

```
164       public void handleSearch() {
165           String query = searchField.getText();
166           MenuCategory cat = filterCategoryBox.getValue();
167           Integer catId = (cat != null && cat.getCategoryId() > 0) ? cat.getCategoryId() : null;
168
169           List<MenuItem> results = menuService.searchAndFilter(query, catId, false);
170           menuTable.setItems(FXCollections.observableArrayList(results));
171       }
172
173       @FXML
```

Component	Method: handleSearch()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 164–173 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 174–183` Method: handleSortByPrice()

```
174       public void handleSortByPrice() {
175           String query = searchField.getText();
176           MenuCategory cat = filterCategoryBox.getValue();
177           Integer catId = (cat != null && cat.getCategoryId() > 0) ? cat.getCategoryId() : null;
178
179           List<MenuItem> results = menuService.searchAndFilter(query, catId, true);
180           menuTable.setItems(FXCollections.observableArrayList(results));
181       }
182
183       @FXML
```

Component	Method: handleSortByPrice()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 174–183 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 184–194` Method: clearForm()

```
184       public void clearForm() {
185           selectedItem = null;
186           nameField.clear();
187           priceField.clear();
188           prepTimeField.clear();
189           descriptionArea.clear();
190           availableCheckBox.setSelected(true);
191           categoryComboBox.getSelectionModel().clearSelection();
192           menuTable.getSelectionModel().clearSelection();
193       }
194   }
```

Component	Method: clearForm()
Line-by-Line Breakdown	Lines 184–194 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

OrderController.java

Path: src\main\java\com\restaurant\controller\OrderController.java | Total Lines: 209 | Logical Blocks: 11

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Order. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–34 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.OrderStatus;
4   import com.restaurant.enums.TableStatus;
5   import com.restaurant.model.Customer;
6   import com.restaurant.model.MenuItem;
7   import com.restaurant.model.Order;
8   import com.restaurant.model.OrderItem;
9   import com.restaurant.model.RestaurantTable;
10  import com.restaurant.model.User;
11  import com.restaurant.service.AuthenticationService;
12  import com.restaurant.service.CustomerService;
13  import com.restaurant.service.MenuService;
14  import com.restaurant.service.OrderService;
15  import com.restaurant.service.TableService;
16  import com.restaurant.util.AlertUtil;
17  import com.restaurant.util.CurrencyUtil;
18  import javafx.collections.FXCollections;
19  import javafx.collections.ObservableList;
20  import javafx.fxml.FXML;
21  import javafx.scene.control.Button;
22  import javafx.scene.control.ComboBox;
23  import javafx.scene.control.Label;
24  import javafx.scene.control.Spinner;
25  import javafx.scene.control.SpinnerValueFactory;
26  import javafx.scene.control.TableColumn;
27  import javafx.scene.control.TableView;
28  import javafx.scene.control.TextArea;
29  import javafx.scene.control.TextField;
30  import javafx.scene.control.cell.PropertyValueFactory;
31
32  import java.util.ArrayList;
33  import java.util.List;
34
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 34. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 35–35 Class Declaration & Heritage

```
35   public class OrderController {
```

Architecture Role	Defines OrderController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 35, allowing cross-package invocation.

Lines 36–71 Class Members & Implementation

```
36
37       @FXML private ComboBox<RestaurantTable> tableComboBox;
38       @FXML private ComboBox<Customer> customerComboBox;
39       @FXML private ComboBox<MenuItem> menuItemComboBox;
40       @FXML private Spinner<Integer> quantitySpinner;
41       @FXML private TextField itemInstructionsField;
42       @FXML private TextArea orderNotesArea;
43
44       @FXML private TableView<OrderItem> currentOrderItemsTable;
45       @FXML private TableColumn<OrderItem, String> colItemName;
46       @FXML private TableColumn<OrderItem, Integer> colItemQty;
47       @FXML private TableColumn<OrderItem, Double> colItemPrice;
48       @FXML private TableColumn<OrderItem, Double> colItemTotal;
49
50       @FXML private Label lblSubtotal;
51       @FXML private Label lblDiscount;
52       @FXML private Label lblTax;
53       @FXML private Label lblServiceCharge;
54       @FXML private Label lblGrandTotal;
55
56       @FXML private TableView<Order> ordersHistoryTable;
57       @FXML private TableColumn<Order, String> colHistNumber;
58       @FXML private TableColumn<Order, String> colHistTable;
59       @FXML private TableColumn<Order, String> colHistCustomer;
60       @FXML private TableColumn<Order, OrderStatus> colHistStatus;
61       @FXML private TableColumn<Order, Double> colHistTotal;
62
63       private final OrderService orderService = new OrderService();
64       private final TableService tableService = new TableService();
65       private final CustomerService customerService = new CustomerService();
66       private final MenuService menuService = new MenuService();
67
68       private final ObservableList<OrderItem> currentItemsList = FXCollections.observableArrayList();
69       private final ObservableList<Order> orderHistoryList = FXCollections.observableArrayList();
70
71       @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 36–71 (36 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–91 Method: initialize()

```
72      public void initialize() {
73          colItemName.setCellValueFactory(new PropertyValueFactory<>("itemName"));
74          colItemQty.setCellValueFactory(new PropertyValueFactory<>("quantity"));
75          colItemPrice.setCellValueFactory(new PropertyValueFactory<>("unitPrice"));
76          colItemTotal.setCellValueFactory(new PropertyValueFactory<>("subtotal"));
77          currentOrderItemsTable.setItems(currentItemsList);
78
79          colHistNumber.setCellValueFactory(new PropertyValueFactory<>("orderNumber"));
80          colHistTable.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
81          colHistCustomer.setCellValueFactory(new PropertyValueFactory<>("customerName"));
82          colHistStatus.setCellValueFactory(new PropertyValueFactory<>("status"));
83          colHistTotal.setCellValueFactory(new PropertyValueFactory<>("totalAmount"));
84
85          quantitySpinner.setValueFactory(new SpinnerValueFactory.IntegerSpinnerValueFactory(1, 50, 1));
86
87          loadDropdowns();
88          loadOrderHistory();
89          recalculateCurrentOrderTotals();
90      }
91
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 72–91 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–97 Method: loadDropdowns()

```
92      private void loadDropdowns() {
93          tableComboBox.setItems(FXCollections.observableArrayList(tableService.getAllTables()));
94          customerComboBox.setItems(FXCollections.observableArrayList(customerService.getAllCustomers
()));
95          menuItemComboBox.setItems(FXCollections.observableArrayList(menuService.getAllMenuItems()));
96      }
97
```

Component	Method: loadDropdowns()
Line-by-Line Breakdown	Lines 92–97 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 98–104` Method: loadOrderHistory()

```
98      private void loadOrderHistory() {
99          orderHistoryList.clear();
100         orderHistoryList.addAll(orderService.getAllOrders());
101         ordersHistoryTable.setItems(orderHistoryList);
102     }
103
104     @FXML
```

Component	Method: loadOrderHistory()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 98–104 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 105–136 Method: handleAddItemToOrder()

```
105       public void handleAddItemToOrder() {
106           MenuItem selectedItem = menuItemComboBox.getValue();
107           if (selectedItem == null) {
108               AlertUtil.showWarning("Item Required", "Please select a menu item to add.");
109               return;
110           }
111
112           int qty = quantitySpinner.getValue();
113           String instructions = itemInstructionsField.getText();
114
115           // Check if item already in order list
116           boolean merged = false;
117           for (OrderItem oi : currentItemsList) {
118               if (oi.getItemId() == selectedItem.getItemId()) {
119                   oi.updateQuantity(oi.getQuantity() + qty);
120                   merged = true;
121                   break;
122               }
123           }
124
125           if (!merged) {
126               OrderItem oi = new OrderItem(selectedItem.getItemId(), selectedItem.getName(), qty, select
edItem.getPrice(), instructions);
127               currentItemsList.add(oi);
128           }
129
130           currentOrderItemsTable.refresh();
131           itemInstructionsField.clear();
132           quantitySpinner.getValueFactory().setValue(1);
133           recalculateCurrentOrderTotals();
134       }
135
136       @FXML
```

Component	Method: handleAddItemToOrder()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 105–136 (32 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 137–144 Method: handleRemoveSelectedItem()

```
137     public void handleRemoveSelectedItem() {
138         OrderItem selected = currentOrderItemsTable.getSelectionModel().getSelectedItem();
139         if (selected != null) {
140             currentItemsList.remove(selected);
141             recalculateCurrentOrderTotals();
142         }
143     }
144
```

Component	Method: handleRemoveSelectedItem()
Line-by-Line Breakdown	Lines 137–144 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 145–160 Method: recalculateCurrentOrderTotals()

```
145     private void recalculateCurrentOrderTotals() {
146         double subtotal = currentItemsList.stream().mapToDouble(OrderItem::getSubtotal).sum();
147         double discount = 0.0;
148         double taxable = subtotal - discount;
149         double tax = taxable * 0.05;
150         double serviceCharge = subtotal * 0.025;
151         double total = taxable + tax + serviceCharge;
152
153         lblSubtotal.setText(CurrencyUtil.format(subtotal));
154         lblDiscount.setText(CurrencyUtil.format(discount));
155         lblTax.setText(CurrencyUtil.format(tax));
156         lblServiceCharge.setText(CurrencyUtil.format(serviceCharge));
157         lblGrandTotal.setText(CurrencyUtil.format(total));
158     }
159
160     @FXML
```

Component	Method: recalculateCurrentOrderTotals()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 145–160 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 161–195 Method: handleCreateAndSendToKitchen()

```
161       public void handleCreateAndSendToKitchen() {
162           if (currentItemsList.isEmpty()) {
163               AlertUtil.showWarning("Empty Order", "Please add at least one menu item to the order.");
164               return;
165           }
166
167           RestaurantTable table = tableComboBox.getValue();
168           Customer customer = customerComboBox.getValue();
169           User currentUser = AuthenticationService.getCurrentUser();
170           int empId = currentUser != null ? currentUser.getUserId() : 1;
171
172           Integer tableId = table != null ? table.getTableId() : null;
173           Integer customerId = customer != null ? customer.getCustomerId() : null;
174
175           try {
176               Order order = orderService.createNewOrder(
177                       customerId,
178                       tableId,
179                       empId,
180                       new ArrayList<>(currentItemsList),
181                       orderNotesArea.getText()
182               );
183
184               AlertUtil.showInfo("Order Placed", "Order #" + order.getOrderNumber() + " created and disp
atched to Kitchen!");
185               currentItemsList.clear();
186               orderNotesArea.clear();
187               recalculateCurrentOrderTotals();
188               loadOrderHistory();
189               loadDropdowns();
190           } catch (Exception e) {
191               AlertUtil.showError("Order Failed", e.getMessage());
192           }
193       }
194
195       @FXML
```

Component	Method: handleCreateAndSendToKitchen()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 161–195 (35 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 196–209` **Method: handleCancelOrder()**

```
196       public void handleCancelOrder() {
197           Order selected = ordersHistoryTable.getSelectionModel().getSelectedItem();
198           if (selected == null) {
199               AlertUtil.showWarning("No Selection", "Please select an order from the history to cance
l.");
200               return;
201           }
202           boolean confirm = AlertUtil.showConfirmation("Cancel Order", "Are you sure you want to cancel
order " + selected.getOrderNumber() + "?");
203           if (confirm) {
204               orderService.cancelOrder(selected.getOrderId());
205               loadOrderHistory();
206               loadDropdowns();
207           }
208       }
209   }
```

Component	Method: handleCancelOrder()
Line-by-Line Breakdown	Lines 196–209 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PaymentController.java

Path: src\main\java\com\restaurant\controller\PaymentController.java | Total Lines: 138 | Logical Blocks: 8

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Payment. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–17 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.OrderStatus;
4   import com.restaurant.enums.PaymentMethod;
5   import com.restaurant.model.*;
6   import com.restaurant.service.AuthenticationService;
7   import com.restaurant.service.OrderService;
8   import com.restaurant.service.PaymentService;
9   import com.restaurant.util.AlertUtil;
10  import com.restaurant.util.CurrencyUtil;
11  import javafx.collections.FXCollections;
12  import javafx.collections.ObservableList;
13  import javafx.fxml.FXML;
14  import javafx.scene.control.*;
15  import javafx.scene.control.cell.PropertyValueFactory;
16  import javafx.scene.layout.VBox;
17
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 17. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 18–18 Class Declaration & Heritage

```
18  public class PaymentController {
```

Architecture Role	Defines PaymentController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 18, allowing cross-package invocation.

Lines 19–50 Class Members & Implementation

```
19
20      @FXML private TableView<Order> unpaidOrdersTable;
21      @FXML private TableColumn<Order, String> colOrderNumber;
22      @FXML private TableColumn<Order, String> colTable;
23      @FXML private TableColumn<Order, String> colCustomer;
24      @FXML private TableColumn<Order, Double> colTotal;
25
26      @FXML private ComboBox<PaymentMethod> paymentMethodCombo;
27      @FXML private Label selectedOrderLabel;
28      @FXML private Label amountDueLabel;
29
30      // Payment specific panels
31      @FXML private VBox cashPanel;
32      @FXML private TextField cashTenderedField;
33      @FXML private Label changeDueLabel;
34
35      @FXML private VBox upiPanel;
36      @FXML private TextField upiIdField;
37      @FXML private TextField upiTxnRefField;
38
39      @FXML private VBox cardPanel;
40      @FXML private TextField cardLastFourField;
41      @FXML private TextField cardAuthCodeField;
42
43      @FXML private TextArea receiptOutputArea;
44
45      private final OrderService orderService = new OrderService();
46      private final PaymentService paymentService = new PaymentService();
47      private final ObservableList<Order> unpaidList = FXCollections.observableArrayList();
48      private Order selectedOrder;
49
50      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 19–50 (32 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 51–74 Method: initialize()

```
51        public void initialize() {
52            colOrderNumber.setCellValueFactory(new PropertyValueFactory<>("orderNumber"));
53            colTable.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
54            colCustomer.setCellValueFactory(new PropertyValueFactory<>("customerName"));
55            colTotal.setCellValueFactory(new PropertyValueFactory<>("totalAmount"));
56
57            paymentMethodCombo.setItems(FXCollections.observableArrayList(PaymentMethod.values()));
58            paymentMethodCombo.setValue(PaymentMethod.CASH);
59
60            paymentMethodCombo.valueProperty().addListener((obs, oldVal, newVal) -> togglePaymentFields(ne
wVal));
61
62            unpaidOrdersTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal)
-> {
63                if (newVal != null) {
64                    selectedOrder = orderService.getOrderById(newVal.getOrderId());
65                    selectedOrderLabel.setText(selectedOrder.getOrderNumber() + " (Table " + (selectedOrde
r.getTableNumber() != null ? selectedOrder.getTableNumber() : "Direct") + ")");
66                    amountDueLabel.setText(CurrencyUtil.format(selectedOrder.getTotalAmount()));
67                    cashTenderedField.setText(String.format("%.2f", selectedOrder.getTotalAmount()));
68                }
69            });
70
71            togglePaymentFields(PaymentMethod.CASH);
72            loadUnpaidOrders();
73        }
74
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 51–74 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–80 Method: togglePaymentFields()

```
75        private void togglePaymentFields(PaymentMethod method) {
76            if (cashPanel != null) cashPanel.setVisible(method == PaymentMethod.CASH);
77            if (upiPanel != null) upiPanel.setVisible(method == PaymentMethod.UPI);
78            if (cardPanel != null) cardPanel.setVisible(method == PaymentMethod.CREDIT_CARD || method == P
aymentMethod.DEBIT_CARD);
79        }
80
```

Component	Method: togglePaymentFields()
Line-by-Line Breakdown	Lines 75–80 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 81–91 Method: loadUnpaidOrders()

```
81      private void loadUnpaidOrders() {
82          unpaidList.clear();
83          for (Order o : orderService.getAllOrders()) {
84              if (o.getStatus() != OrderStatus.COMPLETED && o.getStatus() != OrderStatus.CANCELLED) {
85                  unpaidList.add(o);
86              }
87          }
88          unpaidOrdersTable.setItems(unpaidList);
89      }
90
91      @FXML
```

Component	Method: loadUnpaidOrders()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 81–91 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–136 **Method: handleProcessPayment()**

```
92      public void handleProcessPayment() {
93          if (selectedOrder == null) {
94              AlertUtil.showWarning("Select Order", "Please select an order to process payment.");
95              return;
96          }
97
98          PaymentMethod method = paymentMethodCombo.getValue();
99          User cashier = AuthenticationService.getCurrentUser();
100         int cashierId = cashier != null ? cashier.getUserId() : 1;
101         String invoiceNumber = "INV-" + System.currentTimeMillis() % 10000000;
102         Payment payment;
103
104         // Polymorphic instantiation based on chosen channel
105         switch (method) {
106             case CASH -> {
107                 double tendered = Double.parseDouble(cashTenderedField.getText().trim());
108                 payment = new CashPayment(selectedOrder.getOrderId(), invoiceNumber, selectedOrder.get
TotalAmount(), cashierId, tendered);
109             }
110             case UPI -> {
111                 String upi = upiIdField.getText().trim();
112                 String ref = upiTxnRefField.getText().trim();
113                 payment = new UPIPayment(selectedOrder.getOrderId(), invoiceNumber, selectedOrder.getT
otalAmount(), cashierId, upi, ref);
114             }
115             case CREDIT_CARD, DEBIT_CARD -> {
116                 String lastFour = cardLastFourField.getText().trim();
117                 String auth = cardAuthCodeField.getText().trim();
118                 payment = new CardPayment(selectedOrder.getOrderId(), invoiceNumber, selectedOrder.get
TotalAmount(), method, cashierId, lastFour, auth);
119             }
120             default -> payment = new CashPayment(selectedOrder.getOrderId(), invoiceNumber, selectedOr
der.getTotalAmount(), cashierId, selectedOrder.getTotalAmount());
121         }
122
123         try {
124             boolean success = paymentService.processAndFinalizePayment(payment, selectedOrder);
125             if (success) {
126                 String receipt = payment.generateReceipt(selectedOrder);
127                 receiptOutputArea.setText(receipt);
128                 AlertUtil.showInfo("Payment Successful", "Payment completed successfully!\n" + receip
t);
129
130                 loadUnpaidOrders();
131                 selectedOrder = null;
132                 selectedOrderLabel.setText("None");
133                 amountDueLabel.setText(CurrencyUtil.format(0));
134             }
135         } catch (Exception e) {
136             AlertUtil.showError("Payment Failed", e.getMessage());
137         }
```

Component	Method: handleProcessPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 92–136 (45 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 137–138` Class Members & Continuation

```
137         }
138     }
```

Component	Class Members & Continuation
Line-by-Line Breakdown	Lines 137–138 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ReportController.java

Path: src\main\java\com\restaurant\controller\ReportController.java | Total Lines: 76 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Report. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–12 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.service.ReportService;
4   import javafx.collections.FXCollections;
5   import javafx.collections.ObservableList;
6   import javafx.fxml.FXML;
7   import javafx.scene.chart.BarChart;
8   import javafx.scene.chart.PieChart;
9   import javafx.scene.chart.XYChart;
10
11  import java.util.Map;
12
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 12. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 13–13 Class Declaration & Heritage

```
13  public class ReportController {
```

Architecture Role	Defines ReportController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 13, allowing cross-package invocation.

Lines 14–21 Class Members & Implementation

```
14
15      @FXML private BarChart<String, Number> categorySalesBarChart;
16      @FXML private PieChart paymentMethodsPieChart;
17      @FXML private BarChart<String, Number> topItemsBarChart;
18
19      private final ReportService reportService = new ReportService();
20
21      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 14–21 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 22–25 Method: initialize()

```
22      public void initialize() {
23          loadCharts();
24      }
25
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 22–25 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 26–31 Method: loadCharts()

```
26      public void loadCharts() {
27          loadCategorySalesChart();
28          loadPaymentMethodsChart();
29          loadTopItemsChart();
30      }
31
```

Component	Method: loadCharts()
Line-by-Line Breakdown	Lines 26–31 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 32–45 Method: loadCategorySalesChart()

```
32      private void loadCategorySalesChart() {
33          categorySalesBarChart.getData().clear();
34          Map<String, Double> data = reportService.getSalesByCategory();
35
36          XYChart.Series<String, Number> series = new XYChart.Series<>();
37          series.setName("Revenue (₹)");
38
39          for (Map.Entry<String, Double> entry : data.entrySet()) {
40              series.getData().add(new XYChart.Data<>(entry.getKey(), entry.getValue()));
41          }
42
43          categorySalesBarChart.getData().add(series);
44      }
45
```

Component	Method: loadCategorySalesChart()
Line-by-Line Breakdown	Lines 32–45 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 46–57 Method: loadPaymentMethodsChart()

```
46      private void loadPaymentMethodsChart() {
47          paymentMethodsPieChart.getData().clear();
48          Map<String, Double> data = reportService.getPaymentMethodBreakdown();
49
50          ObservableList<PieChart.Data> pieData = FXCollections.observableArrayList();
51          for (Map.Entry<String, Double> entry : data.entrySet()) {
52              pieData.add(new PieChart.Data(entry.getKey() + " (₹" + entry.getValue() + ")", entry.getVa
lue()));
53          }
54
55          paymentMethodsPieChart.setData(pieData);
56      }
57
```

Component	Method: loadPaymentMethodsChart()
Line-by-Line Breakdown	Lines 46–57 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–72 Method: loadTopItemsChart()

```
58      private void loadTopItemsChart() {
59          topItemsBarChart.getData().clear();
60          Map<String, Integer> data = reportService.getMostPopularItems(5);
61
62          XYChart.Series<String, Number> series = new XYChart.Series<>();
63          series.setName("Quantity Sold");
64
65          for (Map.Entry<String, Integer> entry : data.entrySet()) {
66              series.getData().add(new XYChart.Data<>(entry.getKey(), entry.getValue()));
67          }
68
69          topItemsBarChart.getData().add(series);
70      }
71
72      @FXML
```

Component	Method: loadTopItemsChart()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 58–72 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 73–76 Method: handleRefresh()

```
73      public void handleRefresh() {
74          loadCharts();
75      }
76  }
```

Component	Method: handleRefresh()
Line-by-Line Breakdown	Lines 73–76 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ReservationController.java

Path: src\main\java\com\restaurant\controller\ReservationController.java | Total Lines: 129 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Reservation. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–19 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.ReservationStatus;
4   import com.restaurant.model.Customer;
5   import com.restaurant.model.Reservation;
6   import com.restaurant.model.RestaurantTable;
7   import com.restaurant.service.CustomerService;
8   import com.restaurant.service.ReservationService;
9   import com.restaurant.service.TableService;
10  import com.restaurant.util.AlertUtil;
11  import javafx.collections.FXCollections;
12  import javafx.collections.ObservableList;
13  import javafx.fxml.FXML;
14  import javafx.scene.control.*;
15  import javafx.scene.control.cell.PropertyValueFactory;
16
17  import java.time.LocalDate;
18  import java.time.LocalTime;
19
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 19. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 20–20 Class Declaration & Heritage

```
20  public class ReservationController {
```

Architecture Role	Defines ReservationController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 20, allowing cross-package invocation.

Lines 21–44 Class Members & Implementation

```
21
22      @FXML private TableView<Reservation> reservationTable;
23      @FXML private TableColumn<Reservation, Integer> colId;
24      @FXML private TableColumn<Reservation, String> colCustomer;
25      @FXML private TableColumn<Reservation, String> colPhone;
26      @FXML private TableColumn<Reservation, String> colTable;
27      @FXML private TableColumn<Reservation, LocalDate> colDate;
28      @FXML private TableColumn<Reservation, LocalTime> colTime;
29      @FXML private TableColumn<Reservation, Integer> colGuests;
30      @FXML private TableColumn<Reservation, ReservationStatus> colStatus;
31
32      @FXML private ComboBox<Customer> customerCombo;
33      @FXML private ComboBox<RestaurantTable> tableCombo;
34      @FXML private DatePicker datePicker;
35      @FXML private TextField timeField;
36      @FXML private Spinner<Integer> guestsSpinner;
37      @FXML private TextArea requestArea;
38
39      private final ReservationService reservationService = new ReservationService();
40      private final CustomerService customerService = new CustomerService();
41      private final TableService tableService = new TableService();
42      private final ObservableList<Reservation> reservationList = FXCollections.observableArrayList();
43
44      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 21–44 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 45–62 Method: initialize()

```
45      public void initialize() {
46          colId.setCellValueFactory(new PropertyValueFactory<>("reservationId"));
47          colCustomer.setCellValueFactory(new PropertyValueFactory<>("customerName"));
48          colPhone.setCellValueFactory(new PropertyValueFactory<>("customerPhone"));
49          colTable.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
50          colDate.setCellValueFactory(new PropertyValueFactory<>("reservationDate"));
51          colTime.setCellValueFactory(new PropertyValueFactory<>("reservationTime"));
52          colGuests.setCellValueFactory(new PropertyValueFactory<>("guestCount"));
53          colStatus.setCellValueFactory(new PropertyValueFactory<>("status"));
54
55          datePicker.setValue(LocalDate.now());
56          timeField.setText("19:30");
57          guestsSpinner.setValueFactory(new SpinnerValueFactory.IntegerSpinnerValueFactory(1, 20, 2));
58
59          loadDropdowns();
60          loadReservations();
61      }
62
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 45–62 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–67 Method: loadDropdowns()

```
63      private void loadDropdowns() {
64          customerCombo.setItems(FXCollections.observableArrayList(customerService.getAllCustomers()));
65          tableCombo.setItems(FXCollections.observableArrayList(tableService.getAllTables()));
66      }
67
```

Component	Method: loadDropdowns()
Line-by-Line Breakdown	Lines 63–67 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–74 Method: loadReservations()

```
68      private void loadReservations() {
69          reservationList.clear();
70          reservationList.addAll(reservationService.getAllReservations());
71          reservationTable.setItems(reservationList);
72      }
73
74      @FXML
```

Component	Method: loadReservations()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 68–74 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–108 Method: handleCreateReservation()

```
75        public void handleCreateReservation() {
76            Customer cust = customerCombo.getValue();
77            RestaurantTable table = tableCombo.getValue();
78            LocalDate date = datePicker.getValue();
79            String timeStr = timeField.getText().trim();
80            int guests = guestsSpinner.getValue();
81
82            if (cust == null || table == null || date == null || timeStr.isEmpty()) {
83                AlertUtil.showWarning("Missing Fields", "Please select customer, table, date and time.");
84                return;
85            }
86
87            try {
88                LocalTime time = LocalTime.parse(timeStr.length() == 5 ? timeStr + ":00" : timeStr);
89                boolean created = reservationService.createReservation(
90                        cust.getCustomerId(),
91                        table.getTableId(),
92                        date,
93                        time,
94                        guests,
95                        requestArea.getText()
96                );
97
98                if (created) {
99                    AlertUtil.showInfo("Reserved", "Table #" + table.getTableNumber() + " reserved for " +
cust.getName());
100                   loadReservations();
101                   requestArea.clear();
102               }
103           } catch (Exception e) {
104               AlertUtil.showError("Reservation Error", e.getMessage());
105           }
106       }
107
108       @FXML
```

Component	Method: handleCreateReservation()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 75–108 (34 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 109–119` Method: handleCancelReservation()

```
109       public void handleCancelReservation() {
110           Reservation r = reservationTable.getSelectionModel().getSelectedItem();
111           if (r == null) {
112               AlertUtil.showWarning("No Selection", "Please select a reservation to cancel.");
113               return;
114           }
115           reservationService.updateStatus(r.getReservationId(), ReservationStatus.CANCELLED);
116           loadReservations();
117       }
118
119       @FXML
```

Component	Method: handleCancelReservation()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 109–119 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 120–129` Method: handleCompleteReservation()

```
120       public void handleCompleteReservation() {
121           Reservation r = reservationTable.getSelectionModel().getSelectedItem();
122           if (r == null) {
123               AlertUtil.showWarning("No Selection", "Please select a reservation to complete.");
124               return;
125           }
126           reservationService.updateStatus(r.getReservationId(), ReservationStatus.COMPLETED);
127           loadReservations();
128       }
129   }
```

Component	Method: handleCompleteReservation()
Line-by-Line Breakdown	Lines 120–129 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

SupplierController.java

Path: src\main\java\com\restaurant\controller\SupplierController.java | Total Lines: 137 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Supplier. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.model.Supplier;
4   import com.restaurant.service.SupplierService;
5   import com.restaurant.util.AlertUtil;
6   import javafx.collections.FXCollections;
7   import javafx.collections.ObservableList;
8   import javafx.fxml.FXML;
9   import javafx.scene.control.TableColumn;
10  import javafx.scene.control.TableView;
11  import javafx.scene.control.TextField;
12  import javafx.scene.control.cell.PropertyValueFactory;
13
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public class SupplierController {
```

Architecture Role	Defines SupplierController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–35 Class Members & Implementation

```
15
16      @FXML private TableView<Supplier> supplierTable;
17      @FXML private TableColumn<Supplier, Integer> colId;
18      @FXML private TableColumn<Supplier, String> colName;
19      @FXML private TableColumn<Supplier, String> colContact;
20      @FXML private TableColumn<Supplier, String> colPhone;
21      @FXML private TableColumn<Supplier, String> colEmail;
22      @FXML private TableColumn<Supplier, String> colCategory;
23
24      @FXML private TextField nameField;
25      @FXML private TextField contactField;
26      @FXML private TextField phoneField;
27      @FXML private TextField emailField;
28      @FXML private TextField categoryField;
29      @FXML private TextField addressField;
30
31      private final SupplierService supplierService = new SupplierService();
32      private final ObservableList<Supplier> supplierList = FXCollections.observableArrayList();
33      private Supplier selectedSupplier;
34
35      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 15–35 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 36–58

Method: initialize()

```
36      public void initialize() {
37          colId.setCellValueFactory(new PropertyValueFactory<>("supplierId"));
38          colName.setCellValueFactory(new PropertyValueFactory<>("name"));
39          colContact.setCellValueFactory(new PropertyValueFactory<>("contactPerson"));
40          colPhone.setCellValueFactory(new PropertyValueFactory<>("phone"));
41          colEmail.setCellValueFactory(new PropertyValueFactory<>("email"));
42          colCategory.setCellValueFactory(new PropertyValueFactory<>("suppliedCategory"));
43
44          supplierTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) ->
{
45              if (newVal != null) {
46                  selectedSupplier = newVal;
47                  nameField.setText(newVal.getName());
48                  contactField.setText(newVal.getContactPerson());
49                  phoneField.setText(newVal.getPhone());
50                  emailField.setText(newVal.getEmail());
51                  categoryField.setText(newVal.getSuppliedCategory());
52                  addressField.setText(newVal.getAddress());
53              }
54          });
55
56          loadSuppliers();
57      }
58
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 36–58 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–65

Method: loadSuppliers()

```
59      private void loadSuppliers() {
60          supplierList.clear();
61          supplierList.addAll(supplierService.getAllSuppliers());
62          supplierTable.setItems(supplierList);
63      }
64
65      @FXML
```

Component	Method: loadSuppliers()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 59–65 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–87 Method: handleAddSupplier()

```
66      public void handleAddSupplier() {
67          try {
68              boolean success = supplierService.addSupplier(
69                      nameField.getText(),
70                      contactField.getText(),
71                      phoneField.getText(),
72                      emailField.getText(),
73                      addressField.getText(),
74                      categoryField.getText()
75              );
76
77              if (success) {
78                  AlertUtil.showInfo("Success", "Supplier registered.");
79                  clearForm();
80                  loadSuppliers();
81              }
82          } catch (Exception e) {
83              AlertUtil.showError("Save Error", e.getMessage());
84          }
85      }
86
87      @FXML
```

Component	Method: handleAddSupplier()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 66–87 (22 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–112 Method: handleUpdateSupplier()

```
88      public void handleUpdateSupplier() {
89          if (selectedSupplier == null) {
90              AlertUtil.showWarning("No Selection", "Select a supplier to update.");
91              return;
92          }
93          try {
94              selectedSupplier.setName(nameField.getText());
95              selectedSupplier.setContactPerson(contactField.getText());
96              selectedSupplier.setPhone(phoneField.getText());
97              selectedSupplier.setEmail(emailField.getText());
98              selectedSupplier.setAddress(addressField.getText());
99              selectedSupplier.setSuppliedCategory(categoryField.getText());
100
101             boolean success = supplierService.updateSupplier(selectedSupplier);
102             if (success) {
103                 AlertUtil.showInfo("Success", "Supplier updated.");
104                 clearForm();
105                 loadSuppliers();
106             }
107         } catch (Exception e) {
108             AlertUtil.showError("Update Error", e.getMessage());
109         }
110     }
111
112     @FXML
```

Component	Method: handleUpdateSupplier()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 88–112 (25 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 113–126` Method: handleDeleteSupplier()

```
113       public void handleDeleteSupplier() {
114           if (selectedSupplier == null) {
115               AlertUtil.showWarning("No Selection", "Select a supplier to delete.");
116               return;
117           }
118           boolean confirm = AlertUtil.showConfirmation("Confirm Delete", "Delete " + selectedSupplier.ge
tName() + "?");
119           if (confirm) {
120               supplierService.deleteSupplier(selectedSupplier.getSupplierId());
121               clearForm();
122               loadSuppliers();
123           }
124       }
125
126       @FXML
```

Component	Method: handleDeleteSupplier()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 113–126 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 127–137` Method: clearForm()

```
127       public void clearForm() {
128           selectedSupplier = null;
129           nameField.clear();
130           contactField.clear();
131           phoneField.clear();
132           emailField.clear();
133           categoryField.clear();
134           addressField.clear();
135           supplierTable.getSelectionModel().clearSelection();
136       }
137   }
```

Component	Method: clearForm()
Line-by-Line Breakdown	Lines 127–137 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

TableController.java

Path: src\main\java\com\restaurant\controller\TableController.java | Total Lines: 143 | Logical Blocks: 11

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> JavaFX UI Controller for Table. Manages view rendering, user input validation, button action triggers, and binds observable data collections.

Lines 1–12 Package & Library Imports

```
1   package com.restaurant.controller;
2
3   import com.restaurant.enums.TableStatus;
4   import com.restaurant.model.RestaurantTable;
5   import com.restaurant.service.TableService;
6   import com.restaurant.util.AlertUtil;
7   import javafx.collections.FXCollections;
8   import javafx.collections.ObservableList;
9   import javafx.fxml.FXML;
10  import javafx.scene.control.*;
11  import javafx.scene.control.cell.PropertyValueFactory;
12
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 12. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 13–13 Class Declaration & Heritage

```
13  public class TableController {
```

Architecture Role	Defines TableController as a core component in the Restaurant Management System.
Controller Layer	Acts as the mediator between the JavaFX presentation layer (FXML) and business service layer (MVC Pattern).
Access & Modifiers	Declared with public visibility on line 13, allowing cross-package invocation.

Lines 14–31 Class Members & Implementation

```
14
15      @FXML private TableView<RestaurantTable> tableTable;
16      @FXML private TableColumn<RestaurantTable, Integer> colId;
17      @FXML private TableColumn<RestaurantTable, String> colNumber;
18      @FXML private TableColumn<RestaurantTable, Integer> colCapacity;
19      @FXML private TableColumn<RestaurantTable, String> colLocation;
20      @FXML private TableColumn<RestaurantTable, TableStatus> colStatus;
21
22      @FXML private TextField tableNumberField;
23      @FXML private TextField capacityField;
24      @FXML private TextField locationField;
25      @FXML private ComboBox<TableStatus> statusComboBox;
26
27      private final TableService tableService = new TableService();
28      private final ObservableList<RestaurantTable> tableList = FXCollections.observableArrayList();
29      private RestaurantTable selectedTable;
30
31      @FXML
```

Component	Class Members & Implementation
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 14–31 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 32–54 Method: initialize()

```
32      public void initialize() {
33          colId.setCellValueFactory(new PropertyValueFactory<>("tableId"));
34          colNumber.setCellValueFactory(new PropertyValueFactory<>("tableNumber"));
35          colCapacity.setCellValueFactory(new PropertyValueFactory<>("capacity"));
36          colLocation.setCellValueFactory(new PropertyValueFactory<>("location"));
37          colStatus.setCellValueFactory(new PropertyValueFactory<>("status"));
38
39          statusComboBox.setItems(FXCollections.observableArrayList(TableStatus.values()));
40          statusComboBox.setValue(TableStatus.AVAILABLE);
41
42          tableTable.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {
43              if (newVal != null) {
44                  selectedTable = newVal;
45                  tableNumberField.setText(newVal.getTableNumber());
46                  capacityField.setText(String.valueOf(newVal.getCapacity()));
47                  locationField.setText(newVal.getLocation());
48                  statusComboBox.setValue(newVal.getStatus());
49              }
50          });
51
52          loadTables();
53      }
54
```

Component	Method: initialize()
Lifecycle: initialize()	Executed automatically by JavaFX after all FXML fields are injected. Used to setup cell factories, bind observables, populate dropdowns, and load data.
Line-by-Line Breakdown	Lines 32–54 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 55–61 Method: loadTables()

```
55      private void loadTables() {
56          tableList.clear();
57          tableList.addAll(tableService.getAllTables());
58          tableTable.setItems(tableList);
59      }
60
61      @FXML
```

Component	Method: loadTables()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 55–61 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–78 Method: handleAddTable()

```
62      public void handleAddTable() {
63          try {
64              int capacity = Integer.parseInt(capacityField.getText().trim());
65              boolean success = tableService.addTable(tableNumberField.getText(), capacity, locationFiel
d.getText());
66              if (success) {
67                  AlertUtil.showInfo("Success", "Table created successfully.");
68                  clearForm();
69                  loadTables();
70              }
71          } catch (NumberFormatException e) {
72              AlertUtil.showError("Invalid Input", "Capacity must be an integer.");
73          } catch (Exception e) {
74              AlertUtil.showError("Creation Failed", e.getMessage());
75          }
76      }
77
78      @FXML
```

Component	Method: handleAddTable()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 62–78 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 79–101` Method: handleUpdateTable()

```
79      public void handleUpdateTable() {
80          if (selectedTable == null) {
81              AlertUtil.showWarning("No Selection", "Please select a table to update.");
82              return;
83          }
84          try {
85              selectedTable.setTableNumber(tableNumberField.getText());
86              selectedTable.setCapacity(Integer.parseInt(capacityField.getText().trim()));
87              selectedTable.setLocation(locationField.getText());
88              selectedTable.setStatus(statusComboBox.getValue());
89
90              boolean success = tableService.updateTable(selectedTable);
91              if (success) {
92                  AlertUtil.showInfo("Success", "Table updated successfully.");
93                  clearForm();
94                  loadTables();
95              }
96          } catch (Exception e) {
97              AlertUtil.showError("Update Failed", e.getMessage());
98          }
99      }
100
101     @FXML
```

Component	Method: handleUpdateTable()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 79–101 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 102–118 Method: handleDeleteTable()

```
102       public void handleDeleteTable() {
103           if (selectedTable == null) {
104               AlertUtil.showWarning("No Selection", "Please select a table to delete.");
105               return;
106           }
107           boolean confirm = AlertUtil.showConfirmation("Confirm Delete", "Delete table " + selectedTabl
e.getTableNumber() + "?");
108           if (confirm) {
109               boolean success = tableService.deleteTable(selectedTable.getTableId());
110               if (success) {
111                   AlertUtil.showInfo("Success", "Table removed.");
112                   clearForm();
113                   loadTables();
114               }
115           }
116       }
117
118       @FXML
```

Component	Method: handleDeleteTable()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 102–118 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 119–126 Method: handleSetAvailable()

```
119       public void handleSetAvailable() {
120           if (selectedTable != null) {
121               tableService.changeTableStatus(selectedTable.getTableId(), TableStatus.AVAILABLE);
122               loadTables();
123           }
124       }
125
126       @FXML
```

Component	Method: handleSetAvailable()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 119–126 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 127–134 Method: handleSetCleaning()

```
127       public void handleSetCleaning() {
128           if (selectedTable != null) {
129               tableService.changeTableStatus(selectedTable.getTableId(), TableStatus.CLEANING);
130               loadTables();
131           }
132       }
133
134       @FXML
```

Component	Method: handleSetCleaning()
@FXML Injection	Injects UI controls mapped directly from FXML XML tags. The JavaFX FXMLLoader instantiates and connects these widgets at runtime.
Line-by-Line Breakdown	Lines 127–134 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 135–143 Method: clearForm()

```
135       public void clearForm() {
136           selectedTable = null;
137           tableNumberField.clear();
138           capacityField.clear();
139           locationField.clear();
140           statusComboBox.setValue(TableStatus.AVAILABLE);
141           tableTable.getSelectionModel().clearSelection();
142       }
143   }
```

Component	Method: clearForm()
Line-by-Line Breakdown	Lines 135–143 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.